

Техническое задание

Оптимизация размещения камер наблюдения на дискретной карте здания

1. Постановка задачи

Нужно разместить K камер наблюдения на карте здания так, чтобы покрыть максимум важных точек, не ставить камеры в стены и не создавать лишнее пересечение зон обзора. Задача используется как учебная дискретная постановка для сравнения методов оптимизации.

Поток данных:

data/map.json -> grid, K , R -> optimizer -> coordinates of K cameras -> compute_score -> score

- оптимизируемые переменные - координаты камер $C = [(r_1, c_1), \dots, (r_K, c_K)]$;

- карта и параметры фиксированы в data/map.json;

- все методы сравниваются по одной и той же функции score.

2. Входные данные

Основной входной файл - data/map.json. Формат сделан человекочитаемым: поле grid хранится как список строк, где каждая строка соответствует одной строке сетки.

Поле	Значение	Описание
grid_size	100	размер карты: 100 x 100 клеток
K	10	точное число камер в решении
R	12	радиус обзора камеры в клетках
n_walls	969	число клеток-стен
n_points	150	число важных точек
grid	list[str]	карта здания: одна строка JSON на одну строку сетки

Семантика клеток:

- 0 - свободная клетка; камера может быть размещена здесь;
- 1 - стена;
- 2 - важная точка, которую желательно видеть хотя бы одной камерой.

Координаты задаются как (r, c) , где r - индекс строки, c - индекс столбца. Индексация начинается с нуля, верхний левый угол карты имеет координаты $(0, 0)$.

3. Генерация и вид карты

Карта генерируется модулем src/map_tools/generator.py. Он задает комнаты, дверные проемы и зоны повышенного интереса. Случайность фиксирована seed=42, поэтому карта воспроизводима.

- 70% важных точек размещаются в IMPORTANT_AREAS: верхние кабинеты, переговорная с офисом, серверная;
- 30% важных точек размещаются в остальных свободных клетках;
- стены и дверные проемы задаются явно в коде генератора;
- готовый вход для методов - именно data/map.json, а не сам генератор.

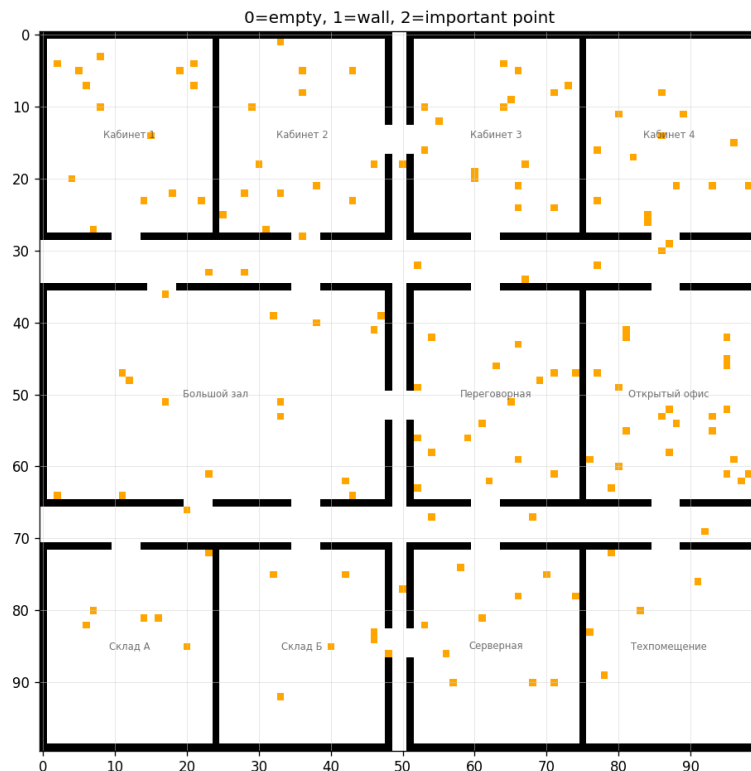


Рисунок 1. Текущая карта здания: белый - свободные клетки, черный - стены, оранжевый - важные точки.

4. Стены и линия обзора

Стена имеет две роли. Во-первых, камера не может стоять в клетке со значением 1. Во-вторых, стена блокирует обзор: камера не видит сквозь стены.

Камера видит клетку (r, c), если одновременно выполнены условия:

- евклидово расстояние от камеры до клетки не больше R;
- между камерой и клеткой есть LOS - line of sight, то есть линия обзора не пересекает стену.

LOS считается функцией `has_los` в `src/scoring.py`: между двумя клетками строится дискретизированный отрезок через `np.linspace`, сэмплы округляются к индексам сетки, затем проверяется отсутствие стены среди этих клеток.

5. Целевая функция

Пусть $C = [(r_1, c_1), \dots, (r_K, c_K)]$ - размещение камер. Нужно максимизировать:

$$\text{Score}(C) = 100 * \text{cov}(C) - 20 * \text{overlap}(C) - 1000 * \text{invalid}(C)$$

- `cov(C)` - число важных точек `grid == 2`, видимых хотя бы одной камерой;
- `overlap(C)` - число клеток карты, видимых двумя или более камерами;
- `invalid(C)` - число камер вне карты или в стене.

Функция `compute_score` в `src/scoring.py` возвращает `score` и `breakdown`: `cov`, `overlap`, `invalid`, `score`. Чем больше `score`, тем лучше решение.

6. Ограничения решения

- метод должен вернуть ровно K координат камер;
- желательно, чтобы все камеры были внутри карты;
- желательно, чтобы все камеры стояли только на свободных клетках 0;
- решение должно покрывать много важных точек и не создавать лишние пересечения обзора;

- invalid-решение формально оценивается, но штраф 1000 за каждую недопустимую камеру делает его плохим.

7. Результат оптимизации

Результат одного запуска метода:

- список из K координат камер;
- итоговый score;
- breakdown: cov, overlap, invalid.

Для честного сравнения все методы используют один и тот же data/map.json и одну и ту же функцию compute_score.

8. Методы оптимизации

Методы оставлены теми же, что в исходном задании. Их нужно реализовать отдельно от генерации карты и scoring. Каждый метод ищет координаты камер и оценивается через compute_score.

Genetic Algorithm

Популяционный метод: каждое решение - список из K камер. Используются отбор лучших решений, crossover и mutation. Возвращает лучший список камер и score.

Particle Swarm Optimization

Каждая частица хранит кандидатное размещение K камер. Движение частицы задается ее личным лучшим решением и глобальным лучшим решением роя. Возвращает лучший список камер и score.

LLM as optimizer

Языковая модель предлагает новые размещения или улучшает существующие кандидаты. compute_score используется как обратная связь: модель получает score и пробует улучшить решение. Возвращает лучший список камер и score.

Важно: методы из scipy.optimize не используются по требованиям docs/task_concept.txt.

9. Проверка зазора для оптимизации

Команда python -m src.verify сравнивает 100 случайных размещений с простым greedy baseline. Greedy добавляет камеры по одной, каждый раз максимизируя прирост покрытия важных точек.

Текущая проверка:

```
random_avg = -2995.60
random_max = 4820
greedy = 7440
gap = 10435.60
```

Так как gap > 5000, у задачи есть заметный зазор: даже простая эвристика сильно лучше случайного размещения, а значит методам оптимизации есть что улучшать и сравнивать.

10. Файлы проекта и команды

Файл	Назначение
data/map.json	основной вход: карта, K, R, статистика
data/map.png	визуализация карты
src/map_tools/generator.py	генерация и загрузка карты
src/map_tools/visualizer.py	визуализация матрицы карты
src/scoring.py	LOS, видимые клетки, compute_score
src/verify.py	проверка random baseline vs greedy baseline
docs/task_concept.txt	исходные требования курса

Основные команды:

```
python -m src.map_tools.generator  
python -m src.map_tools.visualizer  
python -m src.verify  
cat data/map.json | head -20
```

11. Scope модели

Модель является учебной дискретной аппроксимацией. Камеры имеют круговой обзор 360 градусов, высота установки, фокусное расстояние, качество изображения и реальные инженерные нормы не моделируются. Это сделано намеренно, чтобы задача оставалась понятной, воспроизводимой и удобной для сравнения методов оптимизации.